

Angry Santa: a Snowball Simulation (2025/26)

Implementation of a N-body interaction physical process (Author: Grisotto Maurizio)

Project Scope: this project investigates the concepts covered in the “GPU Programming” course through the implementation of a snowball simulation with 3 different applications: CPU Naïve, CPU-Optimized, and GPU. The system models a snowball rolling down an inclined plane and growing by capturing snow particles through probabilistic DEM-inspired contact mechanics as a potential component of a real-time game graphics engine. Both the algorithmic/performance aspects and dynamic Vulkan rendering (GPU version only) were addressed. A theoretical analysis of the multi-GPU extension scenario is also provided.

Physics System Overview: three-subsystem architecture:

- The **snowpack** is a static layer of particles distributed over an inclined plane, arranged on a jittered grid to mimic natural snow. They do not move or collide but can be captured by the rolling ball. After initialization, the snowpack is sorted by increasing X coordinate enabling an optimized binary search that restricts capture checks to only spatially and geometrically relevant particles subset.
- The **shell** consists of dynamic particles forming the outer layer of the snowball, simulated using a simplified Discrete Element Method (DEM: a numerical method in which granular material is represented as a set of discrete particles — small spheres with mass and radius — that interact via pairwise contact forces. When particles overlap, spring + damping forces and friction are applied to separate them, simulating granular behavior without solving continuous equations. Particles interact through collisions, cohesion, and friction, and remain attached to the core via spring-damper constraints). The shell dynamically expands through a capacity-doubling strategy when all particle slots become occupied.
- The **core** is a rigid body with position, velocity, mass, radius, and orientation representing the snowball itself, moving downhill under gravity and drag, growing mass and radius as the shell captures particles.

The probabilistic capture (per thread/particle) algorithm’s overview:

- 1) checks whether the snowpack particle has already been consumed (primary early exit)
- 2) evaluates the particle distance from the current snowball position (second early exit)
- 3) computes a clamped logit score based on parameters: base bias, wetness, relative velocity, ball radius
- 4) generates a deterministic hash random number and compares it against the sigmoid-derived probability

To improve numerical stability and avoid race conditions, three mechanisms are employed: logit clamp, per-frame cap, lock-free atomic reservation of shell slots

Concepts and Techniques:

- Flynn’s Computer Architecture Classification and comparison
- Parallel Algorithm: Fork-Join stream pipeline
- GPU Memory Hierarchy: Registers, Constant, L1/Shared, Read-Only, Texture, DRAM, Pinned
- GPU Optimizations: Memory Coalescing, Shared Memory Tiling, Kernel Fusion, AtomicAdd, `__launch_bounds__`, Occupancy ptxas, hw-optimized math instruction, Warp-Cooperative, CUB Radix Sort, CUDA Streams, Async executions
- Kernel Performance Duration (remote profiling, was limited by system permission constraints)
- Clean and shared code for both CPUs and GPU programs.

Expected Results: measurable GPU speedups over CPUs implementations, stable real-time rendering performance and scalability analysis across different parameters setup and GPU architectures.

Hardware:

Jetson Nano: CPU (SoC) ARM Cortex-A57 (4 cores, 4 threads), GPU Tegra X1 (SM 5.3 – VRAM 4GB), Linux
Laptop: CPU Intel i7-6700HQ (4 cores, 8 threads), GPU GeForce GTX 960M (SM 5.0 – VRAM 4GB), Windows

References: Cundall & Strack (1979): *A discrete numerical model for granular assemblies*; Pöschel & Schwager (2005): *Computational Granular Dynamics*; Green (2010): *Particle Simulation using CUDA*; Anthropic Claude Models (2026): *Opus 4.5, Sonnet 4.5*